

Searching and Sorting!

Process note: I originally posted a giant deck 10-13 so you could see all the final content of the course. After finishing the weeks of Nov 3 and Nov 10, I reorganized all the slides we covered into deck10-11.pdf

This is deck 12-13, all the rest of our content!

Find helpful visualizations here:

<https://visualgo.net/en/sorting>

See also:

Source: [GitHub - vbohush/SortingAlgorithmAnimations](#)

YouTube: [Sorting Algorithm Visualization](#)

A Real-life Example of Sorting

Let's say you want to sort Pokémon cards.
You could sort by HP — how might you do it?

Swapping Pattern

Let's say we want to swap the values at two different spots in a list.

```
temp ← a  
a ← b  
b ← temp
```

(imagine trying to sort a list of physical objects using a single set of tongs)

Selection Sort

Algorithm:

- For every element in the list:
 - Find the smallest element in the rest of the list
 - Swap the current element with that smallest element

Timing Programs

You can use the `time` module to measure how long your program takes.

```
import time
start = time.time()
# your code
end = time.time()
print(end - start)
```

Goodness of an Algorithm

1. **Correctness** — It works.

2. **Desirable qualities:**

- Readable and easy to debug
- Clear and structured
- Robust and maintainable
- Efficient in time and space

Two Ways to Examine "Goodness"

- **Time Complexity** – efficiency of execution
- **Space Complexity** – efficiency of memory use

Efficiency is key for large-scale problems.

Complexity

Amount of time or space required to run an algorithm.

We focus on **time complexity** in this course.

Measuring Time Complexity

We often measure:

- Execution time for a given problem
- Or, number of operations performed (hardware-independent)

Order of an Algorithm

Represents how the number of operations grows with input size n .

Examples: $O(n)$, $O(n^2)$, $O(\log n)$

Standard Reference Functions

Category	Function
Constant	1
Logarithmic	$\log_2(n)$
Linear	n
Linearithmic	$n \log_2(n)$
Quadratic	n^2
Cubic	n^3
Exponential	$a^n, a > 1$

[More on Big O](#)

Example: Calculating Time Complexity

```
x = 0
y = 10
x += 1
for i in range(n):
    x += y
    y += 1000
```

→ $O(n)$ complexity.

Example 2: Nested Loops

```
count = 0
for i in range(n):
    count += 10
    for j in range(n):
        count += j
```

→ $O(n^2)$ complexity.

Best, Worst, and Average Case

- **Best:** fastest scenario
- **Worst:** slowest scenario
- **Average:** expected scenario

Example: Linear Search

- Best: Found at first element
- Worst: Found at last element or not found

Linear search reference:

```
def linear_search(arr, target):  
    """  
    Performs a linear search to find the target value in the given list.  
  
    Args:  
        arr: The list to search within.  
        target: The value to search for.  
  
    Returns:  
        The index of the target if found, otherwise -1.  
    """  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i # Return the index if the target is found  
    return -1 # Return -1 if the target is not found in the list
```

Linear Search Complexity

Case	Complexity
Best	$O(1)$
Worst	$O(n)$

- **Searching:** Imagine looking for a word in a dictionary. If you check one word at a time from the start, it would take ages for a long dictionary. Binary Search makes this faster.
- **Key Idea:** Divide the search space in half each time.

Linear Search vs. Binary Search

- **Linear Search:** $O(n)$
 - Check each element one-by-one.
 - Useful when data is unsorted.
- **Binary Search:** $O(\log n)$
 - Faster, but requires sorted data.
 - Each time, cut the search space in half.

(this will come back) Imagine finding a word in a dictionary. How many words do you skip if you flip halfway each time?

Binary Search Intuition

- **Sorted Array Example:** [1, 3, 5, 7, 9, 11, 13]
 - Target: Find 9.
 - **Step 1:** Start with middle element (7).
 - **Step 2:** Since $9 > 7$, ignore left half.
 - **Step 3:** Now search [9, 11, 13]. Middle is 11.
 - **Step 4:** Since $9 < 11$, ignore right half. Find 9.

Visualization

- **Divide-and-Conquer:**
 - Use visuals to demonstrate the division of the array.
 - Show the array shrinking in size each time.
- **Pseudocode:**

```
low = 0
high = len(array) - 1
while low <= high:
    mid = (low + high) // 2
    if array[mid] == target:
        return mid
    elif array[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
return -1
```

Why Does It Work?

- **Core Concept:** Every decision eliminates half of the possible choices.
- **Comparison to Real Life:** Imagine finding a page in a book without an index. Flip to the middle until you narrow it down.

Practice Problem

- **Problem:** Find the number 23 in the sorted list [5, 12, 15, 23, 27, 30, 35].
- **Thinking out loud:** Can try to write out / verbalize each step.
 - "Is 23 greater or less than the middle element?"
 - Keep cutting until found.

Common Mistakes

- **Off-by-One Errors:** Carefully manage `low` and `high` boundaries.
- **Infinite Loops:** Ensure condition `low <= high`.

Tip: Debugging exercises can help solidify this. Run a failing example and walk through the fix.

Example mistake 1: Incorrect Mid Calculation

```
array = [1, 3, 5, 7, 9, 11, 13]
target = 9

low = 0
high = len(array) - 1
while low <= high:
    mid = (low + high) / 2 # Mistake: Using `/` instead of `//` for integer division
    if array[mid] == target:
        print(f"Found target {target} at index {mid}")
        break
    elif array[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Target not found")
```

Mistake 2: Off-by-One Error

```
# Mistake 2: Off-by-One Error
array = [1, 3, 5, 7, 9, 11, 13]
target = 9

low = 0
high = len(array) # Mistake: Should be `len(array) - 1`
while low <= high:
    mid = (low + high) // 2
    if array[mid] == target:
        print(f"Found target {target} at index {mid}")
        break
    elif array[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Target not found")
```

Example Mistake 3: Infinite Loop Condition

```
array = [1, 3, 5, 7, 9, 11, 13]
target = 9

low = 0
high = len(array) - 1
while low < high: # Mistake: Should be `low <= high`
    mid = (low + high) // 2
    if array[mid] == target:
        print(f"Found target {target} at index {mid}")
        break
    elif array[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Target not found")
```

Fully Worked Example of Binary Search

```
def binary_search(array, target):
    low = 0
    high = len(array) - 1

    while low <= high:
        mid = (low + high) // 2
        # Debug statement to show the current state of low, high, and mid
        print(f"Low: {low}, High: {high}, Mid: {mid}, Mid Value: {array[mid]}")

        if array[mid] == target:
            return mid
        elif array[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1 # Target not found

# Example usage
array = [1, 3, 5, 7, 9, 11, 13]
target = 9
result = binary_search(array, target)
if result != -1:
    print(f"Target {target} found at index {result}")
else:
    print("Target not found")
```

Binary Search Applications

- **Applications:**
 - Finding an element in a sorted array.
 - Searching in a phonebook.
 - Problems like "Guess the Number" (using minimum attempts).

Conclusion & Recap

- **Summary:** Binary Search cuts the search space in half each time, leading to a time complexity of $O(\log n)$.
- **Key Question:** Why does Binary Search need a sorted array?

Engagement: Think of any real-life activities where a binary search-like strategy is used?

<https://www.cs.usfca.edu/~galles/visualization/Search.html>

Logarithmic Complexity

Each iteration halves the problem size $\rightarrow O(\log n)$

Examples:

- Binary search
- Divide and conquer algorithms

Binary Search Analysis

Each split halves the data until 1 element remains.

Height of the tree = $\log_2 n$ comparisons (worst case).

Selection Sort Complexity

(see code in readings: [here](#))

Selection sort compares:

$$n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2 \approx O(n^2)$$

Applies to both best and worst cases.

Comparing Algorithms

Algorithm	Big O
Linear Search	$O(n)$
Binary Search	$O(\log n)$
Selection Sort	$O(n^2)$

Is Sorting Worth It?

Binary search requires sorted data, but sorting costs $O(n^2)$.

Sorting first makes sense when:

- You search many times after sorting.

When n Gets Larger

Algorithm	Big O	Example (n=1000)
Constant	$O(1)$	1
Linear	$O(n)$	1000
Quadratic	$O(n^2)$	1,000,000
Logarithmic	$O(\log n)$	~10

Introducing Merge Sort

Much faster than Selection Sort.

Visualization

Merge Sort Concept

1. Split list until each has one element
2. Merge sorted sublists

Merge Sort Pseudocode

```
def mergeSort(alist):  
    if len(alist) > 1:  
        left = mergeSort(alist[:mid])  
        right = mergeSort(alist[mid:])  
        return merge(left, right)  
    else:  
        return alist
```

Merge Sort Complexity

- Splits → $\log n$
- Merges → n
- **Total: $O(n \log n)$**

Uses extra space for merging.

Review

Algorithm	Worst Case Complexity
Linear Search	$O(n)$
Binary Search	$O(\log n)$
Selection Sort	$O(n^2)$
Merge Sort	$O(n \log n)$

Final Thought

If two algorithms are $O(n)$ and $O(n \log n)$,
and you have a **large dataset**,
choose the $O(n)$ algorithm.

Week 12 Recursion

This folder includes some interactive content we'll work through together during Week 12!

We'll spend some time trying to code key recursive functions and live code them together.

This deck has the SAME content as the readme file in

`other_content/week12_recursion`. I will recommend you open that file during lecture, and I'll project this just for slightly better lecture viewing experience.

Exercise 1: Review tree drawing

First, take a look at `draw_tree.py` in this deck. We're going to talk through how and why this code works. You should copy and paste this into a file on your local machine. Trying running it, then try modifying the *initial function call* so that the tree:

1. Has even more branches
2. Has fewer branches
3. Hits the base case immediately

draw_tree.py

```
import turtle

# Set up the turtle screen
screen = turtle.Screen()
screen.bgcolor("white")
screen.title("Recursive Tree")

# Set up the turtle
tree_turtle = turtle.Turtle()
#tree_turtle.shape("turtle")
tree_turtle.speed("fastest") # Speed up the drawing

# Recursive function to draw a fractal tree
def draw_tree(turtle, branch_length, angle):
    if branch_length > 5: # Base case: stop when the branch is too short
        # Draw the main branch
        turtle.forward(branch_length)

        # Right branch
        turtle.right(angle)
        draw_tree(turtle, branch_length - 15, angle)

        # Left branch
        turtle.left(2 * angle)
        draw_tree(turtle, branch_length - 15, angle)

        # Reset position and angle for next branch
        turtle.right(angle)
        turtle.backward(branch_length)

# Initial position
tree_turtle.left(90) # Point the turtle upwards
tree_turtle.penup()
tree_turtle.goto(0, -200) # Move to starting position
tree_turtle.pendown()

# Draw the tree
draw_tree(tree_turtle, branch_length=100, angle=20)

# Finish up
turtle.done()
```


Exercise 2: Recursive Factorial

Problem: Write a recursive function that returns the factorial of a number. The factorial of n is equal to n times the factorial of $n-1$.

Start with a single parameter function definition: `def factorial(n):`

Examples:

- $0! = 1$ (important: the factorial of 0 is equal to 1.)
- $1! = 1$
- $2! = 2 * 1 = 2 * 1!$
- $3! = 3 * 2 * 1 = 3 * 2!$
- so, `factorial(4) # -> 24`

► Hint 1: Base Case

► Hint 2: Recursive Case

► Solution

Exercise 3: Sums with recursion

Exercise 3a: Summing all integers from 1 to n

Write a recursive function in Python, `recursive_sum`, that calculates the sum of all integers from 1 up to a given positive integer `n`.

Start with a single parameter function definition: `def recursive_sum(n):`

Examples:

- `recursive_sum(4)` # Output: 10 (1 + 2 + 3 + 4)
- `recursive_sum(5)` # Output: 15 (1 + 2 + 3 + 4 + 5)

► Hint 1: Base Case

► Hint 2: Recursive Case

► Solution

Exercise 3b: Summing all integers in a list

Problem: Write a recursive function in Python, `recursive_sum_list`, that calculates the sum of all integers in a given list.

Start with a single parameter function definition: `def recursive_sum_list(numbers):`

Examples:

- `recursive_sum_list([1, 2, 3, 4])` # Output: 10
- `recursive_sum_list([5, 7, 3])` # Output: 15

► Hint 1: Base Case

► Hint 2: Recursive Case

► Hint 3: List Indexing

► Solution

Exercise 4: Reverse a string

Write a recursive function in Python, `recursive_reverse`, that reverses a given string. Start with a single parameter function definition: `def recursive_reverse(s):`

Example:

- `recursive_reverse("hello")` # Output: "olleh"
- `recursive_reverse("abc")` # Output: "cba"

- ▶ Hint 1: Base Case
- ▶ Hint 2: Recursive Case
- ▶ Hint 3: String Indexing

► Solution

Exercise 5: Palindrome checker

Write a recursive function in Python, `is_palindrome`, that checks if a given string is a palindrome (reads the same forward and backward). Start with a single parameter function definition: `def is_palindrome(s):`

Examples:

- `is_palindrome("racecar")` # Output: True
- `is_palindrome("hello")` # Output: False

► Hint 1: Base Case

► Hint 2: Recursive Case

► Hint 3: String Indexing

► Solution

Exercise 6: Challenge!

"Nested List Sum"

Problem: Write a recursive function in Python, `nested_sum`, that calculates the sum of all integers in a list. However, this time, some elements of the list might be nested lists themselves. The function should recursively process these nested lists to calculate the total sum.

You can use `isinstance(item, int)` to check an item is an integer and `isinstance(item, list)` to check if an item is a list.

Start with a single parameter function definition: `def nested_sum(data):`

Examples:

- `nested_sum([1, 2, [3, 4], 5])` # Output: 15
- `nested_sum([[1, 2, [3]], 4, [5, [6]]])` # Output: 21

► Solution

Exercise 7: Count Vowels in a String

Problem: Write a recursive function, `count_vowels`, that counts the number of vowels (a, e, i, o, u) in a given string. The function should be case-insensitive, so both uppercase and lowercase vowels are counted.

Start with a single parameter function definition: `def count_vowels(s):`

Examples:

- `count_vowels("hello") # Output: 2`
- `count_vowels("Recursion is fun!") # Output: 6`
- `count_vowels("xyz") # Output: 0`

► Hint 1: Base Case

► Hint 2: Recursive Case

► Solution

The Big Review Deck



From Priscilla Du Preez @ <https://unsplash.com/photos/scenery-of-bridge-in-front-of-silhouette-of-mountain-Vm0nC-VKFTc>

Week 1: Introduction to CS and Problem Solving

- Know that CS is problem-solving
- Problem solving by subdividing tasks
- Characteristics of an algorithm
- Plagiarism rules for programming
- What is pseudocode?
- Write Python comments, program header block info
- Output "Hello World" using print()
- Programming languages vs. natural languages
- Use IDLE and an IDE
- Submit code as .zip or .py file

Week 2: Basics of Python Programming

- Design/plan algorithms using pseudocode/comments
- Input from terminal to a variable or without saving
- Assign a value to a variable
- Output a string variable, string concatenation
- Variable naming rules
- Data types introduction (focus on String)
- Lists and `random.choice()` (including imports)
- Purpose of modules, imports at top
- If/elif/else, logical operators (`and` , `or`)
- Comparison operators in conditional statements
- Boolean expressions, testing programs interactively

Week 3: Strings, Lists, and Conditionals

- String methods: `strip()`, `lower()`, `upper()`
- Identify String data type
- Test methods using REPL
- Use `in` keyword with strings and lists
- Create a list from variables/user inputs
- Loop over list elements with `for`
- Understand `range()` function and index variable
- Integer type and conversion to String
- Concatenation rules, nested conditionals
- Error types: syntax vs. semantic
- Method chaining with `.`

Week 4: Loops and Arithmetic

- Use `range()` with variables - Loop to reduce code duplication - Integers, floats, initialization - Accumulator pattern, `+=` shortcut - Get list length, convert input to Integer
- Division types, arithmetic operations - Print floats to given decimals

```
# Accumulator Pattern and Loop with Range
total = 0
for i in range(1, 6): # Loop from 1 to 5
    total += i # Accumulate the sum

print(f"Sum of numbers from 1 to 5 is: {total}")

# Basic arithmetic operations
div_result = 10 / 3 # Division (float)
print(f"Division result: {div_result:.2f}") # Print to 2 decimal places
```

Week 5: File I/O, Lists, and Comparisons

- Read lines from text files
- String splitting, list indexing/slicing
- Compare numbers and strings
- operator precedence (light treatment)
- Nested conditionals, nested loops
- find common elements between lists
- List concatenation, accumulation pattern for strings/lists
- Calculate max/min, coordination between lists by index

```
# Reading a File and Splitting Lines into Lists
with open("example.txt", "r") as file:
    for line in file:
        words = line.strip().split() # Split line into words
        print(words) # List of words
```

Week 7: Bits, Bytes, and Data Representation

- `len()` function, string immutability, loops by index
- Bits and bytes, converting bytes to bits
- Binary to decimal, hexadecimal goals
- ASCII and Unicode overview
- RGB storage in hexadecimal
- Important numbers: 1 byte = 8 bits, 8-bit per character for 8-bit character set, 6 hexadecimal digits in an RGB number, therefore 24 bits, because 6 digits x 4 bits per digit
- Different numbering systems (base-2, base-10, base-16) require different number of "symbols"

```
# Convert Bytes to Bits and Decimal to Binary
bytes_value = 4
```

Week 8: Turtle Graphics and Functions

- Use Turtle package for drawing (specific methods that are "testable" to be described in class)
- Read Turtle code and understand the purpose
- Create and call functions (with/without parameters)
- Loop variable as function argument
- Turtle coloring using RGB values
- Variable scope (function vs. global)

```
# Turtle Graphics Drawing a Square
import turtle

def draw_square(size):
    pen = turtle.Turtle()
    for _ in range(4):
        pen.forward(size)
```

Week 9: Functions and While Loops

- Define and use functions, `return` vs `print()`
- Effect of `return` in loops, handling `None`
- Identify appropriate use of `while` vs `for`
- Sentinel and multiple control variables in `while`
- Use `while` for input validation

```
# Function that Returns a Value and While Loop for Validation
def double_number(num):
    return num * 2

valid_input = False
while not valid_input:
    user_input = input("Enter a positive number: ")
    if user_input.isdigit() and int(user_input) > 0:
        valid_input = True
        result = double_number(int(user_input))
```

Week 10: Modules and Image Processing

- Create and use custom modules
- Module shortnames and relationship of modules to filesystem
- RGB colors and pixel manipulation
- Access/modify 2D/3D lists for images
- Using course-specific `cmpt120images` module for images
- Return Boolean expressions directly

```
# Custom Module Import and RGB Color Manipulation
import cmpt120images

image = cmpt120images.loadImage("example.png")
height = len(image)
width = len(image[0])

for y in range(height):
```

Week 11: Lists, Aliases, and Recursion

- Alias vs. copy of lists
- Function parameter and argument as aliases
- List modification inside functions
- Basic recursion, draw recursive patterns
- 3 characteristics of recursive function
- Write factorial, sum of list, reverse string using recursion
- Check if a string is a palindrome (recursive/iterative)

```
# Recursive Function to Calculate Factorial
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```


Week 12: Search Algorithms

- Review recursion
- Write linear search functions (Boolean/index/indices)
- Recognize and code recursive binary search

```
# Binary Search Implementation
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

```
numbers = [1, 3, 5, 7, 9, 11]
index = binary_search(numbers, 7)
```

Week 13: Sorting and Complexity

- Swap list elements, selection sort
- Intermediate steps of selection sort
- Measure runtime with `time`
- Apply algorithms on data from a file
- Characteristics of a good algorithm
- Big O notation and complexity examples. 7 reference functions.
- Merge Sort approach and complexity
- Critical operations, best/worst case scenarios
- Analyze binary search and merge two lists

```
# Selection Sort Implementation
def selection_sort(arr):
    n = len(arr)
```